HTML-CSS-02 第2回_まとめ

第2回の目標

- **新しいCSSの知識**
    - 要素を固定する方法を知る
    - 要素の重なり順を扱うCSSを知る
    - 擬似要素を作れるようになる
- **コーディングを効率化する**
    - 複数クラスを使えるようになる
    - 似たようなレイアウトを探して楽にコーディングをしよう（下層ページを作る）

position

positionプロパティは、要素の「位置」を指定する際に使用する。特に以下の場面で用いられる - 要素を自由に配置したい - 要素の重なり順を変えたい - ヘッダーを固定して表示したい

指定できる値

- static(初期値)：
    - 位置を動かすことはできない
- relative
    - 位置の基準としたい要素に指定することが多い。
    - absoluteとセットで使用
    - 距離指定のプロパティを設定すれば、位置調整が可能
    - z-indexで重なり順を指定する事が可能
- absolute
    - 基準となる要素（`position`が指定された直近の祖先要素）からの距離で、絶対的な位置を決める
    - 親要素を`relative`（または`fixed`）にすることで、親要素を基準に、距離指定のプロパティ（`top`/`left`など）で設定した位置に表示される
    - 元々配置されていた場所からはその要素の高さが無くなる（一段上に浮く）
    - `z-index`で重なり順を指定することが可能
- fixed
    - 画面の決まった位置に要素を固定する。スクロールしても要素が固定され、追従した状態で表示される

    - 指定方法・特徴は`absolute`とほぼ同じと考えてよい
    - 基準がwindow（画面）全体になるので注意。画面左上からどれだけの距離に配置するかを考えて位置を指定する
- sticky
    - 「粘着」という意味の通り、スクロールした際に要素をシールのように粘着させ、固定表示できる
    - 親要素と子要素を用意し、子要素に`sticky`を指定する。親要素を**sticky container**、子要素を**sticky item**と呼ぶ
    - 特徴
        - 親要素を用意することで、固定する（追従する）範囲を指定できる
        - 追従が開始されると`fixed`のように要素が固定され、指定した位置（`top`など）に留まり続ける
        - 親要素の表示範囲を超えると、追従が解除されて通常の配置（static）に戻る
    - 弱点として、ブラウザによって挙動が異なる・使えないケースがあるので、 can I useで対応状況を確認しておくとよい

relativeとabsoluteの関係

- 「ステージ（`relative`）と推し（`absolute`）」のような関係。自分が指定したステージの上で、推しを好きな位置に置ける
- `absolute`の要素に使う`top`・`bottom`・`left`・`right`は、`relative`をつけた要素（ステージ）の大きさを基準にした移動距離
- `transform: translate()`は、つけた要素**自身**の大きさを基準にした移動距離

```
/* .kvを基準に、.kv_contentsを上から40%・左から20%の位置に置く */
.kv {
  position: relative;
}

.kv_contents {
  position: absolute;
  top: 40%;
  left: 20%;
}
```

stickyとfixedの違い

値	高さ	固定される範囲
`sticky`	自身の高さが残るので、他の要素の配置に影響しない	親要素の中で追従する

値	高さ	固定される範囲
`fixed`	高さを失い、下の要素が上にずれ込む	画面自体に固定するイメージ

- `sticky`は必ず`top`とセットで使う
- position: fixed;で要素を固定する方法（stickyとの違い）

実践例①：本文＋サイドバー（sticky container / sticky item）

```
<div class="sticky_wrapper">
  <div class="sticky_container">
    <main class="sticky_main">
      <!-- 記事本文など、縦に長いコンテンツ -->
    </main>

    <aside class="sticky_aside">
      <!-- スクロールに追従させたい要素（広告・目次など） -->
    </aside>
  </div>
</div>
```

```
.sticky_aside {
  position: sticky;
  top: 48px; /* 画面上端から48pxの位置で止まる */
}
```

- `.sticky_container`：sticky containerにあたる。子要素の追従できる範囲＝この要素の高さ分になる
- `.sticky_aside`：sticky itemにあたる。`.sticky_main`（記事本文）より要素の高さが低いため、スクロールしても途中まで画面に留まり続け、`.sticky_container`の下端に到達すると追従が終了する

実践例②：ヘッダーの固定

ヘッダーをスクロールしても画面上部で固定表示させたい、というのも典型的な`sticky`の使いどころ。

```
.header {
  position: sticky;
  top: 0;
  left: 0;
```

```
  z-index: 100;
}
```

- ポイント：`sticky`は親要素の中でのみ固定される。`top`プロパティをつけ忘れると機能しない
- `z-index`を指定しておくことで、他の要素の下に隠れず常に前面に表示される
- `sticky`を使うことで、要素が浮いてスタイルが崩れるのを防ぐことができる（`fixed`だと元の位置から要素が抜けてレイアウトが崩れやすい）

positionまとめ表

値	基準	重なり順(z-index)	主な用途
static	なし（初期値）	指定不可	通常の配置
relative	自分自身の元の位置	指定可能	absoluteの基準を作る
absolute	position指定のある直近の親要素	指定可能	要素を自由な位置に配置する
fixed	ブラウザの画面（viewport）	指定可能	要素を常に同じ位置に固定したい時（ただし元の位置から浮くのでレイアウト崩れに注意）
sticky	親要素（sticky container）の範囲内	指定可能	スクロールに合わせて一定範囲だけ追従させたい要素（ヘッダー固定など）

z-index（重なり順）

`position`が`static`以外の要素に対して、重なり順を数値で指定するプロパティ。数値が大きいほど手前に表示される。

```
.header {
  position: sticky;
  top: 0;
  z-index: 100;
}
```

- `position`が`static`（初期値）のままだと`z-index`は効かない（`position`に`z-index`が必須というわけではない）
- ヘッダーは下に潜ると困るので、大きい数値をつける
- マイナスの値（`z-index: -1;`）にすると、他の要素より後ろに回せる（例：見出しの裏に薄い英字を置く）

- 親要素どうしの重なり順の中で子要素の重なり順が決まる仕組み（**スタッキングコンテキスト**）があるため、子要素の`z-index`をいくら大きくしても、親要素ごと下にある場合は手前に出てこないことがある
- 値がバラバラにならないよう、第4回ではCSS変数（`--z-index-header`など）で一覧管理する
- z-index（MDN）
- スタッキングコンテキストを理解する

ページネーション

ブログサイトの記事一覧では、1ページに表示される記事の数に制限を設けている。制限を超えた記事がある他のページへ移動するときに使うのが**ページネーション**。

```
<div class="pagination">
  <span class="pagination_numbers pagination_numbers__current">1</span>
  <a href="" class="pagination_numbers">2</a>
  <span class="pagination_numbers pagination_numbers__between">…</span>
  <a href="" class="pagination_numbers">10</a>
  <a href="" class="pagination_numbers pagination_numbers__next"></a>
</div>
```

```
.pagination {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 8px; /* flexアイテム同士の間隔をまとめて指定できる */
}

.pagination_numbers {
  width: 40px;
  height: 40px;
  display: flex;
  align-items: center;
  justify-content: center;
  border: 2px solid #f7f7f7;
}

.pagination_numbers__next {
  position: relative;
}
```

```
/* 「>」の矢印を疑似要素で作る */
.pagination_numbers__next::before {
  content: ""; /* contentは疑似要素に必須！ */
  width: 8px;
  height: 8px;
  border-top: 1px solid #333;
  border-right: 1px solid #333;
  position: absolute;
  top: 50%;
  left: 32%;
  transform: translateY(-50%) rotate(45deg);
}
```

- 矢印は、正方形の「上」と「右」の2辺だけに線を引き、`rotate(45deg)`で45度回転させて作る
- `transform`は`translateY(-50%) rotate(45deg)`のように、スペース区切りで複数の関数を同時に指定できる（左から順に適用される）
- 共通の見た目（枠線・大きさ）は`.pagination_numbers`、一部だけ違う見た目（現在のページ・矢印）は`__current`／`__next`のクラスでまとめている（後述の「複数クラスの指定」を参照）
- `gap`は親要素（flexコンテナ）に指定し、子要素同士の余白をまとめて作れる。`margin`と違い、端の要素に余計な余白がつかない

aタグとspanタグの使い分け

タグ	使う場面
`a`	ページ遷移する（クリックできる）もの
`span`	ページ遷移しない（クリックできない）もの（現在のページ、「…」など）

- `span`や`a`はデフォルトで`display: inline;`なので、`width`・`height`が効かない。今回は上下左右中央寄せのために`display: flex;`をあてたことで、効くようになっている（ inline・block・inline-blockの違い）
- `display: flex;`は横並びだけでなく、四角の中の文字を上下左右中央に置く用途でも使う。`align-items`・`justify-content`は`display: flex;`とセットでないと効かない
- 実際にページを切り替える機能は、WordPressで作る。HTMLはWordPressが自動で出力するので、CSSだけつければよい

矢印を擬似要素で作る理由

- どのブラウザでも同じ形で綺麗に見せられる

- HTMLに無駄な記述を減らせる
    - ブラウザ（検索エンジン）はHTMLの構造を見て評価するので、構造が適切でないと評価を下げる原因になる（SEO対策）
    - スクリーンリーダーで閲覧する人に、正しく情報が伝わらなくなるのを防ぐ
- 矢印は「正方形を作る → 角の2辺に枠線をつける → 回転させる」→「`relative` / `absolute` で配置 → `top` ・ `left` ・ `transform` で調整」の順で作る
- 位置の数値は、検証ツールを見ながら目分量で決めてよい（ Chromeのデベロッパーツールの使い方）
- 三角アイコンと矢印アイコンをつくる

疑似要素（::before / ::after）

HTML側に要素を追加することなく、CSSだけである要素の「直前」「直後」に新しい要素があるかのようにコンテンツを追加できる仕組み。

書式

```
セレクタ::before {
  content: "追加したい内容"; /* contentプロパティは必須。装飾だけの場合は空文字でOK */
  /* その他のプロパティ */
}

セレクタ::after {
  content: "追加したい内容";
}
```

- `::before`：対象要素の中身の「一番最初」に生成される
- `::after`：対象要素の中身の「一番最後」に生成される
- `content` プロパティを指定しないと、疑似要素は生成されない（画面に表示されない）
- 生成された疑似要素は、通常の要素と同じようにCSS（`position`、`width`、`color` など）で装飾できる
- 図形を作るときは `content: "";` と空にするが、値に文字を入れると、その文字が擬似要素として表示される

```
/* 見出しの裏に、薄い英字を表示する */
.pseudo_txt {
  position: relative;
}

.pseudo_txt::after {
```

```
  content: "Feature";               /* 表示する文字 */
  font-size: 60px;
  font-weight: bold;
  color: rgba(255, 145, 88, 0.2);   /* 薄い色にする */
  position: absolute;
  top: -50px;
  left: 50%;
  transform: translateX(-50%);      /* 左右中央に置く */
  z-index: -1;                      /* 見出しの後ろに回す */
}
```

- CSSの疑似要素とは？beforeとafterの使い方／疑似クラスと疑似要素、:と::の違い

transform / transition

transform

要素の見た目（移動・回転・拡大縮小・傾斜）を変形させるCSSプロパティ。要素そのものの配置（`position`など）は変えず、見た目上だけを変形させる。

関数	効果	記述例
`translate(x, y)`	移動させる	`transform: translate(30px, 50px);`
`rotate(角度)`	回転させる	`transform: rotate(45deg);`
`scale(倍率)`	拡大・縮小させる	`transform: scale(1.2);`
`skewX(角度)` / `skewY(角度)`	傾斜させる	`transform: skewX(15deg);`

transition

プロパティの値が変化する際に、変化の過程をアニメーションのように滑らかにするCSSプロパティ。`:hover`などの状態変化と組み合わせてよく使う。

```
transition: プロパティ名 タイミング関数 時間;
```

- `transition`は「これから変化するかもしれないプロパティ」を持つ要素（変化前の状態）に指定する
- `:hover`時の見た目（変化後の状態）に`transform`を指定することで、ホバーした瞬間に指定した時間をかけて滑らかに変形する

- `:hover`側に`transition`を書くと、マウスを離したときの戻りのアニメーションがうまくいかないので注意（☝ transitionの使い方）
- 共通する指定（「1秒かけて変化」など）は、共通のクラスにまとめて書くとシンプルになる
- `transform`は1つのセレクタに1行しか書けない。複数の変形は`transform: translateY(-50%) rotate(45deg);`のように1行にまとめる
- 拡大は`perspective()`と`translateZ()`でもできるが、基本は`scale`の方がシンプル

```
.transform_item {
  transition: transform ease-out 1s; /* 変化前の要素に書く */
}

.transform_item__1:hover { transform: translate(30px, 50px); } /* 移動 */
.transform_item__2:hover { transform: rotate(45deg); }        /* 回転 */
.transform_item__3:hover { transform: scale(1.2); }           /* 拡大 */
.transform_item__4:hover { transform: skewX(15deg); }         /* 傾斜 */
```

- 【＋α】hoverやpositionでできるアニメーション：☝ CSSマウスホバーアニメーション16選／ホバーで線が動くアニメーション／スクロールを促すラインアニメーション

複数クラスの指定

1つの要素には、クラスをスペース区切りで複数指定できる。

```
<span class="pagination_numbers pagination_numbers__current">1</span>
```

- `pagination_numbers`：その種類の要素全体に共通するベースのスタイル（枠線・大きさなど）をまとめるクラス
- `pagination_numbers__current`：その中でも「現在のページだけ」といった一部の要素にだけ当てたい差分のスタイルをまとめるクラス

このように「共通スタイルをまとめるベースクラス」と「差分だけを追加するクラス」を役割分担すると、スタイルの重複を減らせる（第4回のBEMのModifierの考え方にも近い）。

- 詳細度（セレクタの優先度）はクラスセレクタ同士であれば基本的に同じなので、どちらが優先されるかはCSSファイル内で「後に書かれている方」で決まる。HTML側でクラスをどの順番で書くかは関係ない。
- ☝ CSSセレクタとは？よく使うセレクタやタグの指定方法をわかりやすく解説！

下層ページを作るときの考え方

トップページ（`index.html`）以外の各ページ（カテゴリーページ・個別ページなど）のことを、まとめて**下層ページ**と呼ぶ。

基本方針

- ヘッダー・フッターなど、ページ間で共通のパーツはコピー＆ペーストで使い回す
- 他ページへのリンクは、`href`属性内に相対パスで記述する（ 相対パスと絶対パスの違いと書き方を学ぼう）
- ページ固有の要素には、ページ名を接頭辞にしたクラス名をつける（第4回参照）
    - 例：`new.html`というページ固有の見出しなら`.new-title`のように命名する
- よく使い回されるパーツ：ヘッダー、フッター、ボタン、見出し、タグ
- サイドバーは`aside`タグで作る（ページの補足的な情報を示すタグ。 HTMLタグ一覧（asideタグ））
- 下層ページでも`head`タグ（Google Fontsの読み込みなど）を忘れずに書く

広い範囲をコピーする方法

- 方法①：コピーしたい範囲の始まりの行をクリック → Shiftを押しながら終わりの行をクリック
- 方法②：行番号の横の「▶」でタグを折りたたみ、開始タグと終了タグの2行だけを選んでコピー（中身もすべてコピーされる）

子孫セレクタでまとめて指定する

記事本体のように、中身（見出しや本文）に直接クラス名をつけられない場面では、親要素のクラスを使って子孫セレクタでまとめて指定する方法もある。

```
/* .articleの中にあるh1だけにスタイルを当てる */
.article h1 {
  font-size: 22px;
  font-weight: bold;
}
```

CMS化（WordPressなど）した際、記事本文中の見出しには直接クラス名をつけられないことが多いため、この書き方に慣れておくと実務でも役立つ。

ダミーテキストを用意する

個別ページの本文のように、まだ中身が決まっていない文章は、ダミーテキストを入れてレイアウトを確認する。 - ダミーテキストジェネレーター

今日の授業のポイント

1. `position: sticky;` は要素を固定できる（親要素の範囲内でのみ）
2. `z-index` は要素の重なり順を指定できるプロパティ
3. 位置調整したい要素に `position: absolute;` 、基準としたい要素に `position: relative;` を使う
4. 擬似要素の使い方：`::before` と `::after` の2種類があり、 `content` プロパティが必須
5. 複数クラスを使うことで、「共通スタイル」と「差分スタイル」に役割分担できる
6. 下層ページは、共通部分をコピーし、差分だけを作り込むと効率がいい

参考記事

- CSSのpositionプロパティで要素を配置しよう
- stickyを使って要素を固定しよう
- CSSのtransformプロパティで要素を変形させよう
- CSSの擬似要素で擬似的に要素を追加しよう
- CSSのtransitionプロパティでアニメーションをかけよう
- CSSセレクタとは？よく使うセレクタやタグの指定方法をわかりやすく解説！
- 相対パスと絶対パスの違いと書き方を学ぼう
- can I use（sticky）
- z-index（MDN）
- スタッキングコンテキストを理解する
- ダミーテキストジェネレーター

よく使うCSSプロパティ

- `color` ：テキストの文字色を指定
  - キーワードによる指定：「black」「white」「red」「blue」など、色名を直接指定する
    - 例：`color: red;`
  - 16進数（HEX）による指定：`#` に続けて16進数で指定する
    - 例：`color: #ff0000;`
  - RGB / RGBA値による指定：赤・緑・青の数値や透明度で指定する
    - 例：`color: rgb(255, 0, 0);`
    - 例：`color: rgba(255, 0, 0, 0.5);`
- `font-size` ：テキストの大きさを指定
  - px（ピクセル）による指定：固定サイズで指定する
    - 例：`font-size: 16px;`
  - rem / em（相対値）による指定：ルートや親要素のサイズを基準とした割合で指定する

    - 例：`font-size: 1.5rem;`
- %（パーセント）による指定：親要素の文字サイズに対する割合で指定する
    - 例：`font-size: 100%;`